

```

import React, { useState, useRef, useEffect } from 'react';
import { Camera, Check, ArrowRight, Mail, Loader2, Crosshair, X, Plus } from 'luc

export default function App() {
  const [screen, setScreen] = useState('landing');
  const [capturedImage, setCapturedImage] = useState(null);
  const [analyzing, setAnalyzing] = useState(false);
  const [analysisStep, setAnalysisStep] = useState(0);
  const [matchedShade, setMatchedShade] = useState(null);
  const [samplePoints, setSamplePoints] = useState([]); // array of {x, y} taps
  const [email, setEmail] = useState('');
  const [waitlistJoined, setWaitlistJoined] = useState(false);
  const [waitlistCount, setWaitlistCount] = useState(1);
  const [rootShade, setRootShade] = useState(null);
  const [needsAtelierSystem, setNeedsAtelierSystem] = useState(false);
  const [tryItOnActive, setTryItOnActive] = useState(false);
  const fileInputRef = useRef(null);
  const imgRef = useRef(null);

  useEffect(() => {
    const loadCount = async () => {
      try {
        const result = await window.storage?.get('waitlist_count', true);
        if (result?.value) setWaitlistCount(parseInt(result.value));
      } catch (e) { /* default */ }
    };
    loadCount();
  }, []);

  // MUSEUM-GRADE MINIMALIST PALETTE
  // Warm off-white, near-black ink, single restrained gold accent
  const c = {
    paper: '#fbfaf6',          // Warm off-white (gallery wall)
    paperShadow: '#f3f1e9',    // Subtle paper depth
    bone: '#e8e3d6',           // Soft hairline
    silver: '#b5ae9e',          // Refined neutral mid
    pewter: '#7a7468',          // Deep neutral
    ink: '#1a1715',             // Near-black warm
    inkDeep: '#0a0907',         // Deepest black
    gold: '#9c7e4a',            // Restrained gold accent
    goldHover: '#b89460',       // Lighter gold for hover
    pop: '#ff3d7f',             // High-contrast pink – for crosshair only, stands
  };
};

```

```
const PRODUCT_IMAGE_DATA_URL = "
```

```
const basePigments = [  
  // Lights & neutrals  
  { id: 'p1', name: 'Blanc Pur', hex: '#f5f0e8', rgb: [245, 240, 232] },  
  { id: 'p2', name: 'Argent', hex: '#b8b4ad', rgb: [184, 180, 173] },  
  { id: 'p3', name: 'Or Pâle', hex: '#e6d5a8', rgb: [230, 213, 168] },  
  { id: 'p14', name: 'Fumée', hex: '#5a5550', rgb: [90, 85, 80] },  
  // Browns  
  { id: 'p6', name: 'Miel', hex: '#a07848', rgb: [160, 120, 72] },  
  { id: 'p9', name: 'Châtaigne', hex: '#5c3a21', rgb: [92, 58, 33] },  
  { id: 'p10', name: 'Marron Froid', hex: '#4a3828', rgb: [74, 56, 40] },  
  { id: 'p11', name: 'Espresso', hex: '#2c1f17', rgb: [44, 31, 23] },  
  { id: 'p12', name: 'Noir Vrai', hex: '#0f0a08', rgb: [15, 10, 8] },  
  // Warm tones  
  { id: 'p7', name: 'Cuivre', hex: '#9a4a28', rgb: [154, 74, 40] },  
  { id: 'p8', name: 'Cramoisi', hex: '#7a2820', rgb: [122, 40, 32] },  
  { id: 'p15', name: 'Ambré', hex: '#d96a2c', rgb: [217, 106, 44] },  
  { id: 'p16', name: 'Topaze', hex: '#c89030', rgb: [200, 144, 48] },  
  // Pinks & roses  
  { id: 'p4', name: 'Rose Thé', hex: '#d8a8a0', rgb: [216, 168, 160] },  
  { id: 'p17', name: 'Pétale', hex: '#e898b0', rgb: [232, 152, 176] },  
  { id: 'p18', name: 'Fuchsia', hex: '#c44878', rgb: [196, 72, 120] },  
  { id: 'p19', name: 'Corail', hex: '#e87878', rgb: [232, 120, 120] },  
  // Cools – purples, violets  
  { id: 'p5', name: 'Lilas', hex: '#b0a8c4', rgb: [176, 168, 196] },  
  { id: 'p13', name: 'Violette', hex: '#3a2a4a', rgb: [58, 42, 74] },  
  { id: 'p20', name: 'Lavande', hex: '#b89cd0', rgb: [184, 156, 208] },  
  { id: 'p21', name: 'Améthyste', hex: '#6a3c9a', rgb: [106, 60, 154] },  
  // Blues  
  { id: 'p22', name: 'Ciel', hex: '#9ec0d8', rgb: [158, 192, 216] },  
  { id: 'p23', name: 'Lagon', hex: '#3c7a8c', rgb: [60, 122, 140] },  
  { id: 'p24', name: 'Indigo', hex: '#2a3a6a', rgb: [42, 58, 106] },  
  { id: 'p25', name: 'Saphir', hex: '#4a6cc8', rgb: [74, 108, 200] },  
  // Greens  
  { id: 'p26', name: 'Sauge', hex: '#9ab098', rgb: [154, 176, 152] },  
  { id: 'p27', name: 'Émeraude', hex: '#2c5a3a', rgb: [44, 90, 58] },  
  { id: 'p28', name: 'Brume', hex: '#a8d4b8', rgb: [168, 212, 184] },  
];
```

```
// ===== Real CV functions =====
```

```
const rgbToLab = (r, g, b) => {  
  let [rN, gN, bN] = [r / 255, g / 255, b / 255];  
  rN = rN > 0.04045 ? Math.pow((rN + 0.055) / 1.055, 2.4) : rN / 12.92;  
  gN = gN > 0.04045 ? Math.pow((gN + 0.055) / 1.055, 2.4) : gN / 12.92;  
  bN = bN > 0.04045 ? Math.pow((bN + 0.055) / 1.055, 2.4) : bN / 12.92;  
  const x = (rN * 0.4124 + gN * 0.3576 + bN * 0.1805) * 100;
```

```

const y = (rN * 0.2126 + gN * 0.7152 + bN * 0.0722) * 100;
const z = (rN * 0.0193 + gN * 0.1192 + bN * 0.9505) * 100;
const xN = x / 95.047, yN = y / 100, zN = z / 108.883;
const fx = xN > 0.008856 ? Math.pow(xN, 1/3) : (7.787 * xN) + 16/116;
const fy = yN > 0.008856 ? Math.pow(yN, 1/3) : (7.787 * yN) + 16/116;
const fz = zN > 0.008856 ? Math.pow(zN, 1/3) : (7.787 * zN) + 16/116;
return {
  L: Math.round((116 * fy) - 16),
  a: Math.round(500 * (fx - fy)),
  b: Math.round(200 * (fy - fz)),
};
};

const deltaE = (lab1, lab2) => Math.sqrt(
  Math.pow(lab1.L - lab2.L, 2) + Math.pow(lab1.a - lab2.a, 2) + Math.pow(lab1.b
);

// Detects if a colour is likely skin (warm orange-pink tones with high lightne
// Used to filter out skin pixels accidentally caught in hair samples
const isLikelySkin = (r, g, b) => {
  const total = r + g + b;
  if (total < 200) return false; // too dark to be skin
  // Skin typically: r > g > b, with r-g and g-b both small-positive
  const rgGap = r - g;
  const gbGap = g - b;
  if (rgGap < 5 || rgGap > 60) return false;
  if (gbGap < 5 || gbGap > 50) return false;
  return r > 140 && r < 240; // typical skin red range
};

// Compute median of a 1D array
const median = (arr) => {
  const sorted = [...arr].sort((a, b) => a - b);
  const mid = Math.floor(sorted.length / 2);
  return sorted.length % 2 === 0
    ? (sorted[mid - 1] + sorted[mid]) / 2
    : sorted[mid];
};

// FIX 1 + FIX 2: Multi-point sampling with median + outlier rejection
//
// The function now accepts ARRAY of sample points (so customer can tap multipl
// Each tap area is widened to 200x200 (vs old 80x80) for more representative s
// Across all collected pixels:
//   - Outliers rejected (extreme highlights, deep shadows, skin tones)
//   - Final colour computed via median per channel (more robust than k-means d
// This gives a "dimensional" match that represents the hair's true mid-tone, n

```

```

//
// PHASE 2 / PRODUCTION NOTE:
// For true hair-only painting and segmentation (Fix 3), a developer should int
// pretrained ML hair-segmentation model (e.g., MediaPipe Selfie Segmentation e
// hair, or a custom TensorFlow.js model). That work requires ~5-15MB model wei
// performance tuning, and is beyond this prototype's scope.
const extractDominantColour = (imageEl, points) => {
  const canvas = document.createElement('canvas');
  canvas.width = imageEl.naturalWidth;
  canvas.height = imageEl.naturalHeight;
  const ctx = canvas.getContext('2d');
  ctx.drawImage(imageEl, 0, 0);

  // Aggregate samples from every tap point
  const allSamples = [];
  for (const point of points) {
    const cx = Math.round((point.x / 100) * canvas.width);
    const cy = Math.round((point.y / 100) * canvas.height);
    // Wider sample area - 200px or 1/4 of canvas, whichever smaller
    const sampleSize = Math.min(200, Math.floor(canvas.width / 4));
    const sx = Math.max(0, cx - sampleSize / 2);
    const sy = Math.max(0, cy - sampleSize / 2);
    const sw = Math.min(sampleSize, canvas.width - sx);
    const sh = Math.min(sampleSize, canvas.height - sy);
    const pixels = ctx.getImageData(sx, sy, sw, sh).data;
    for (let i = 0; i < pixels.length; i += 4) {
      const r = pixels[i], g = pixels[i+1], b = pixels[i+2];
      const brightness = (r + g + b) / 3;
      // Reject extreme highlights (too bright) and shadows (too dark)
      if (brightness < 25 || brightness > 235) continue;
      // Reject likely skin tones
      if (isLikelySkin(r, g, b)) continue;
      allSamples.push([r, g, b]);
    }
  }
  if (allSamples.length === 0) return [128, 128, 128];

  // Compute median per channel (more robust than mean - ignores outliers natur
  const rValues = allSamples.map(s => s[0]);
  const gValues = allSamples.map(s => s[1]);
  const bValues = allSamples.map(s => s[2]);
  const medianR = median(rValues);
  const medianG = median(gValues);
  const medianB = median(bValues);

  // Final outlier-rejection pass: reject samples very far from the median, the
  // This gives a "trimmed mean" that's robust to remaining outliers but still

```

```

const trimmed = allSamples.filter(([r, g, b]) => {
  const dist = Math.sqrt(
    Math.pow(r - medianR, 2) +
    Math.pow(g - medianG, 2) +
    Math.pow(b - medianB, 2)
  );
  return dist < 60; // within 60 RGB units of median
});
const finalSamples = trimmed.length > 0 ? trimmed : allSamples;
const avg = finalSamples.reduce((acc, p) => [acc[0]+p[0], acc[1]+p[1], acc[2]
return [
  Math.round(avg[0] / finalSamples.length),
  Math.round(avg[1] / finalSamples.length),
  Math.round(avg[2] / finalSamples.length),
];
};

const generateRecipe = (targetRgb) => {
  const targetLab = rgbToLab(targetRgb[0], targetRgb[1], targetRgb[2]);
  let bestRecipe = null;
  let bestDistance = Infinity;
  for (let i = 0; i < basePigments.length; i++) {
    for (let j = i + 1; j < basePigments.length; j++) {
      for (let k = j + 1; k < basePigments.length; k++) {
        for (let a = 10; a <= 90; a += 10) {
          for (let b = 5; b <= 90 - a; b += 10) {
            const cVal = 100 - a - b;
            if (cVal < 5) continue;
            const blendR = (basePigments[i].rgb[0] * a + basePigments[j].rgb[0]
            const blendG = (basePigments[i].rgb[1] * a + basePigments[j].rgb[1]
            const blendB = (basePigments[i].rgb[2] * a + basePigments[j].rgb[2]
            const blendLab = rgbToLab(blendR, blendG, blendB);
            const dist = deltaE(targetLab, blendLab);
            if (dist < bestDistance) {
              bestDistance = dist;
              bestRecipe = [
                { ...basePigments[i], pct: a },
                { ...basePigments[j], pct: b },
                { ...basePigments[k], pct: cVal },
              ].sort((x, y) => y.pct - x.pct);
            }
          }
        }
      }
    }
  }
  return { recipe: bestRecipe, deltaE: bestDistance };
}

```

```

};

const getUndertone = (lab) => lab.b > 12 ? 'Warm' : lab.b < 6 ? 'Cool' : 'Neutr
const getConfidence = (dE) => {
  if (dE < 2) return 99;
  if (dE < 5) return Math.round(98 - dE);
  if (dE < 10) return Math.round(95 - dE / 2);
  return Math.max(70, Math.round(90 - dE));
};

// Convert RGB to HSL for proper hue detection
const rgbToHsl = (r, g, b) => {
  r /= 255; g /= 255; b /= 255;
  const max = Math.max(r, g, b), min = Math.min(r, g, b);
  let h, s, l = (max + min) / 2;
  if (max === min) {
    h = s = 0;
  } else {
    const d = max - min;
    s = l > 0.5 ? d / (2 - max - min) : d / (max + min);
    switch (max) {
      case r: h = ((g - b) / d + (g < b ? 6 : 0)) * 60; break;
      case g: h = ((b - r) / d + 2) * 60; break;
      case b: h = ((r - g) / d + 4) * 60; break;
    }
  }
  return { h, s, l };
};

const getShadeName = (lab, undertone, rgb) => {
  const [r, g, bl] = rgb || [0, 0, 0];
  const hsl = rgbToHsl(r, g, bl);
  const { h, s, l } = hsl;

  // CRITICAL: Lower saturation threshold to 0.15 – real photo pixels are less
  // than pure colour swatches due to lighting, shadows, and clustering averagi
  // Brown hair has saturation around 0.3-0.5 BUT its hue sits in 20-40° (orang
  // so we need a more careful test: high saturation OR distinct non-warm hue.

  const isWarmHue = h >= 15 && h < 50; // Orange-yellow region (where br
  const isVibrantWarm = isWarmHue && s > 0.55; // Truly bright orange/yellow vs
  const isCoolHue = h >= 50 && h < 345; // Greens, cyans, blues, purples
  const isVibrantCool = isCoolHue && s > 0.15; // Most cool fashion colours

  // ===== MAISON HUE SIGNATURE SHADE NAMING =====
  // Naming language: poetic, jewel-toned, single evocative words
  // Each shade reads like a perfume name from a luxury house

```

```

// VIBRANT WARM (orange/yellow/red beyond hair tones)
if (isVibrantWarm) {
    if (h >= 35 && h < 50) return l > 0.55 ? 'Solène' : 'Topaze';      // golde
    if (h >= 15 && h < 35) {
        if (l > 0.65) return 'Pêche';          // soft peach
        if (l > 0.45) return 'Ambré';          // warm amber-orange
        return 'Cuivre Vif';                    // burnt copper
    }
}

// VIBRANT COOL (everything that isn't a warm hair tone)
if (isVibrantCool) {
    // RED (very narrow band, only when truly red not auburn)
    if ((h < 15 || h >= 345) && s > 0.5) {
        if (l > 0.55) return 'Cerise';          // bright cherry red
        return 'Rubis';                          // deep ruby
    }
    // GREEN (50-165°)
    if (h >= 50 && h < 165) {
        if (h < 70) return l > 0.6 ? 'Citron' : 'Olivine';    // yellow-green
        if (l > 0.65) return 'Brume';                    // pale mint
        if (l > 0.4) return 'Sauge';                      // sage
        return 'Émeraude';                                // deep emerald
    }
    // CYAN-TEAL (165-200°)
    if (h >= 165 && h < 200) {
        if (l > 0.6) return 'Aqua';
        return 'Lagon';                                  // deep teal
    }
    // BLUE (200-260°)
    if (h >= 200 && h < 260) {
        if (l > 0.65) return 'Ciel';                    // soft sky
        if (l > 0.45) return 'Saphir';                  // sapphire
        return 'Nuit';                                    // deep night blue
    }
    // PURPLE (260-300°)
    if (h >= 260 && h < 300) {
        if (l > 0.65) return 'Lilas';                    // soft lilac
        if (l > 0.45) return 'Iris';                      // mid violet
        return 'Améthyste';                                // deep amethyst
    }
    // PINK / MAGENTA (300-345°)
    if (h >= 300 && h < 345) {
        if (l > 0.75) return 'Pétale';                    // palest pink (baby pink)
        if (l > 0.6) return 'Églantine';                  // luminous rose pink
        if (l > 0.4) return 'Fuchsia';                    // vivid magenta
    }
}

```

```

    return 'Grenade'; // deep pomegranate
  }
}

// SOFT PINK detection – for baby pinks that may register as low saturation b
// This catches the case where pink reads as "low saturation but L is high an
if (l > 0.7 && h >= 300 && h < 360 && s > 0.05) {
  return 'Pétale'; // baby pink rescue
}
if (l > 0.7 && h >= 0 && h < 30 && s > 0.05 && r > g && r > bl) {
  return 'Pêche'; // peach rescue
}

// ===== NATURAL HAIR COLOURS =====
// Maison Hue signature names – French/poetic, distinctive

const L = lab.L, a = lab.a, b = lab.b;

if (L > 80) return undertone === 'Warm' ? 'Champagne' : 'Perle';
if (L > 65) return undertone === 'Warm' ? 'Miel' : 'Platine';
if (L > 50) {
  if (a > 15) return 'Rose Thé'; // tea rose blonde
  if (b > 25) return 'Caramel';
  return 'Brume Chaude'; // warm mist (mushroom)
}
if (L > 35) {
  if (a > 20) return 'Cuivre'; // copper
  if (a > 12 && b > 15) return 'Acajou'; // mahogany auburn
  return 'Marron Glacé'; // glazed chestnut
}
if (L > 20) {
  if (a > 8) return 'Sable';
  return 'Espresso';
}
return undertone === 'Cool' ? 'Obsidienne' : 'Onyx';
};

const getDescription = (name) => ({
  // ===== NATURAL – Maison Hue Signature =====
  'Champagne': 'Soft beige with golden warmth',
  'Perle': 'Luminous white with cool clarity',
  'Miel': 'Honeyed caramel, sun-kissed',
  'Platine': 'Cool, icy blonde',
  'Rose Thé': 'Tea-rose blonde, delicate',
  'Caramel': 'Warm, golden brunette',
  'Brume Chaude': 'Warm mist, soft taupe',
  'Cuivre': 'Vivid russet brilliance',

```



```

'Acajou': 'Mahogany red-brown',
'Marron Glacé': 'Glazed chestnut depth',
'Sable': 'Deep cocoa with warmth',
'Espresso': 'Rich coffee-bean depth',
'Obsidienne': 'Deepest violet-black',
'Onyx': 'True dimensional black',
// ===== WARM TONES =====
'Solène': 'Luminous golden yellow',
'Topaze': 'Deep honeyed gold',
'Pêche': 'Soft luminous peach',
'Ambré': 'Warm amber-orange',
'Cuivre Vif': 'Burnt copper, vivid',
'Cerise': 'Bright cherry red',
'Rubis': 'Deep ruby red',
// ===== GREENS =====
'Citron': 'Fresh lemon-green',
'Olivine': 'Warm yellow-green',
'Brume': 'Pale mint mist',
'Sauge': 'Soft botanical green',
'Émeraude': 'Deep emerald jewel',
// ===== BLUES =====
'Aqua': 'Bright tropical cyan',
'Lagon': 'Deep oceanic teal',
'Ciel': 'Soft sky blue',
'Saphir': 'Sapphire blue',
'Nuit': 'Deep midnight blue',
// ===== PURPLES =====
'Lilas': 'Soft lilac dream',
'Iris': 'Mid violet',
'Améthyste': 'Deep amethyst purple',
// ===== PINKS =====
'Pétale': 'Palest rose petal',
'Églantine': 'Luminous wild rose',
'Fuchsia': 'Vivid magenta',
'Grenade': 'Deep pomegranate pink',
}[name] || 'A bespoke Maison Hue shade'));

const analysisStages = [
  'Reading image data',
  'Sampling root region',
  'Clustering pigment values',
  'Computing LAB co-ordinates',
  'Generating your formula',
];

// Root shade options – AI determines length colour from the photo
const rootOptions = [

```

```

    { id: 'r1', label: 'Very light', sub: 'Natural blonde, white, or grey', hex:
    { id: 'r2', label: 'Light to medium brown', sub: '', hex: '#8a6a4a' },
    { id: 'r3', label: 'Dark brown', sub: '', hex: '#3d2817' },
    { id: 'r4', label: 'Black', sub: '', hex: '#0f0a08' },
    { id: 'r5', label: 'Not sure', sub: 'Our AI will determine', hex: null },
  ];

const proceedFromHairInfo = () => {
  if (rootShade) {
    // Show photo guidance screen first, then upload
    setScreen('photoGuide');
  }
};

const handleCapture = (e) => {
  const file = e.target.files?.[0];
  if (!file) return;
  if (!file.type.startsWith('image/')) {
    alert('Please upload a photograph (JPG, PNG, or HEIC).');
    return;
  }
  const reader = new FileReader();
  reader.onload = (ev) => {
    setCapturedImage(ev.target.result);
    setSamplePoints([]); // start with no taps – customer must tap
    setScreen('aim');
  };
  reader.onerror = () => {
    alert('Sorry – there was a problem reading your photograph. Please try anot');
  };
  reader.readAsDataURL(file);
};

const startCapture = () => fileInputRef.current?.click();
const runRealAnalysis = () => { setScreen('analyze'); setAnalyzing(true); setAn

useEffect(() => {
  if (analyzing && analysisStep < analysisStages.length) {
    const timer = setTimeout(() => setAnalysisStep(s => s + 1), 800);
    return () => clearTimeout(timer);
  } else if (analyzing && analysisStep >= analysisStages.length && imgRef.curre
    try {
      const dominantRgb = extractDominantColour(imgRef.current, samplePoints);
      const lab = rgbToLab(dominantRgb[0], dominantRgb[1], dominantRgb[2]);
      const undertone = getUndertone(lab);
      const { recipe, deltaE: dE } = generateRecipe(dominantRgb);
      const confidence = getConfidence(dE);

```

```

const name = getShadeName(lab, undertone, dominantRgb);
const hex = '#' + dominantRgb.map(v => v.toString(16).padStart(2, '0')).join('');
const formulaNum = Math.floor(Math.random() * 900 + 100);

// Detect if Atelier System is needed:
// Map customer's stated root darkness to an approximate L value, then compare
// If the match is lighter than the roots, light pigment alone won't cover
// The Veil whitening base is required first.
const rootApproxL = {
  'r1': 80, // very light (blonde/grey/white)
  'r2': 55, // light-medium brown
  'r3': 30, // dark brown
  'r4': 12, // black
  'r5': 50, // not sure - assume medium
}[rootShade] || 50;

// Veil needed if: roots are dark-ish AND the match is meaningfully light
// OR: the matched colour is a fashion colour (any saturated non-natural)
const matchL = lab.L;
const matchIsLighter = matchL > rootApproxL + 15; // 15 L units of lift required
const rootsAreDark = rootApproxL < 50;
const atelierNeeded = rootsAreDark && matchIsLighter;
setNeedsAtelierSystem(atelierNeeded);

setTimeout(() => {
  setMatchedShade({
    name, hex, rgb: dominantRgb, lab, undertone,
    fullRecipe: recipe, confidence,
    deltaE: dE.toFixed(2),
    desc: getDescription(name),
    formulaNum,
  });
  setAnalyzing(false);
  setScreen('result');
}, 600);
} catch (err) {
  console.error('CV error:', err);
  setAnalyzing(false);
}
}, [analyzing, analysisStep, samplePoints]);

const joinWaitlist = async () => {
  // Proper email validation - must have something@something.something
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!email || !emailRegex.test(email)) return;
  try {
    const entry = {

```

```

    email,
    timestamp: new Date().toISOString(),
    matchedShade: matchedShade?.name || null,
    hex: matchedShade?.hex || null,
    formulaNum: matchedShade?.formulaNum || null,
    rootShade,
    needsAtelierSystem,
  };

  await window.storage?.set(`waitlist:${Date.now()}`, JSON.stringify(entry),
  const newCount = waitlistCount + 1;
  await window.storage?.set('waitlist_count', String(newCount), true);
  setWaitlistCount(newCount);
} catch (e) { /* fail gracefully */ }
setWaitlistJoined(true);
};

// MH monogram – refined, museum-quality
const MHMark = ({ size = 28, color = c.ink, strokeWidth = 0.75 }) => (
  <svg width={size} height={size * 1.05} viewBox="0 0 50 52" fill="none">
    <path d="M 7 8 L 7 44" stroke={color} strokeWidth={strokeWidth} />
    <path d="M 7 8 L 18 26 L 29 8" stroke={color} strokeWidth={strokeWidth} fil
    <path d="M 29 8 L 29 44" stroke={color} strokeWidth={strokeWidth} />
    <path d="M 29 26 L 43 26" stroke={color} strokeWidth={strokeWidth} />
    <path d="M 43 8 L 43 44" stroke={color} strokeWidth={strokeWidth} />
  </svg>
);

// PreviewCanvas – paints matched colour only on the top portion of the head (r
// Properly simulates The Veil whitening base before colour for dark-to-light t
const PreviewCanvas = ({ imageSrc, matchedRgb, sampleX, sampleY, useVeil = fals
  const canvasRef = useRef(null);
  useEffect(() => {
    if (!imageSrc || !matchedRgb || !canvasRef.current) return;
    const img = new Image();
    img.crossOrigin = 'anonymous';
    img.onload = () => {
      const canvas = canvasRef.current;
      if (!canvas) return;
      const maxDim = 800;
      const scale = Math.min(1, maxDim / Math.max(img.naturalWidth, img.natural
      canvas.width = Math.round(img.naturalWidth * scale);
      canvas.height = Math.round(img.naturalHeight * scale);
      const ctx = canvas.getContext('2d');
      ctx.drawImage(img, 0, 0, canvas.width, canvas.height);
      const imgData = ctx.getImageData(0, 0, canvas.width, canvas.height);
      const data = imgData.data;
      const W = canvas.width;

```

```

const H = canvas.height;

// Sample original colour at the tap point (used for hair pixel detection)
const cx = Math.round((sampleX / 100) * W);
const cy = Math.round((sampleY / 100) * H);
const sampleR = 25;
const samples = [];
for (let dy = -sampleR; dy <= sampleR; dy++) {
  for (let dx = -sampleR; dx <= sampleR; dx++) {
    const px = cx + dx, py = cy + dy;
    if (px < 0 || py < 0 || px >= W || py >= H) continue;
    const i = (py * W + px) * 4;
    samples.push([data[i], data[i+1], data[i+2]]);
  }
}
if (samples.length === 0) return;
const avgOriginal = samples.reduce((acc, s) => [acc[0]+s[0], acc[1]+s[1],
  .map(v => v / samples.length);

// ROOT REGION CONSTRAINT —
// Paint only the TOP portion of the image (where roots/parting always ar
// Strong opacity in the top 40% of the image, fading out smoothly betwee
// Below 55% from the top, no painting occurs (this is the lengths/face/b
const fullStrengthBottom = H * 0.40; // 0% to 40% from top: full strength
const fadeOutBottom = H * 0.55;      // 40% to 55%: fade out
// Below 55%: no painting

const tolerance = 75; // hair-pixel detection tolerance — wider for more

for (let py = 0; py < H; py++) {
  // 1. Compute vertical region mask
  let regionMask;
  if (py < fullStrengthBottom) {
    regionMask = 1;
  } else if (py < fadeOutBottom) {
    const fade = (py - fullStrengthBottom) / (fadeOutBottom - fullStrengt
    regionMask = 1 - fade;
  } else {
    continue; // below the fade zone — skip entirely
  }

  for (let px = 0; px < W; px++) {
    const i = (py * W + px) * 4;
    const r = data[i], g = data[i+1], b = data[i+2];

    // 2. Hair pixel detection — is this pixel similar to the sampled hai
    const colourDist = Math.sqrt(

```

```

    Math.pow(r - avgOriginal[0], 2) +
    Math.pow(g - avgOriginal[1], 2) +
    Math.pow(b - avgOriginal[2], 2)
);
if (colourDist > tolerance) continue; // not hair - skip

// 3. Closeness factor (stronger blend on closer-matching pixels)
const closeness = 1 - (colourDist / tolerance);

// 4. Compute target colour with optional Veil step
const origLuma = 0.299 * r + 0.587 * g + 0.114 * b;
const avgLuma = 0.299 * avgOriginal[0] + 0.587 * avgOriginal[1] + 0.114 * avgOriginal[2];
const lumaDelta = (origLuma - avgLuma); // preserves shadow/highlight

let targetR, targetG, targetB;

if (useVeil) {
    // ATELIER SYSTEM:
    // Step 1 - VEIL: lift dark pixels almost completely to warm white.
    // This is what makes light/fashion colour visible on dark hair.
    const veilColour = [248, 244, 234]; // warm off-white, very pale
    const veilStrength = 0.97; // near-full lift
    const veiledR = r * (1 - veilStrength) + veilColour[0] * veilStrength;
    const veiledG = g * (1 - veilStrength) + veilColour[1] * veilStrength;
    const veiledB = b * (1 - veilStrength) + veilColour[2] * veilStrength;

    // Step 2 - HUE: apply matched colour over the veiled base at full strength
    const colourStrength = 0.95;
    targetR = veiledR * (1 - colourStrength) + matchedRgb[0] * colourStrength;
    targetG = veiledG * (1 - colourStrength) + matchedRgb[1] * colourStrength;
    targetB = veiledB * (1 - colourStrength) + matchedRgb[2] * colourStrength;
} else {
    // STANDARD: directly apply colour at high strength
    const colourStrength = 0.92;
    targetR = r * (1 - colourStrength) + matchedRgb[0] * colourStrength;
    targetG = g * (1 - colourStrength) + matchedRgb[1] * colourStrength;
    targetB = b * (1 - colourStrength) + matchedRgb[2] * colourStrength;
}

// 5. Re-add a small amount of texture variation
const textureShift = lumaDelta * 0.20;
targetR = Math.max(0, Math.min(255, targetR + textureShift));
targetG = Math.max(0, Math.min(255, targetG + textureShift));
targetB = Math.max(0, Math.min(255, targetB + textureShift));

// 6. Final blend = closeness * regionMask
const finalBlend = closeness * regionMask;

```

```

        data[i]    = Math.round(r * (1 - finalBlend) + targetR * finalBlend);
        data[i+1] = Math.round(g * (1 - finalBlend) + targetG * finalBlend);
        data[i+2] = Math.round(b * (1 - finalBlend) + targetB * finalBlend);
    }
}
ctx.putImageData(imgData, 0, 0);
};
img.src = imageSrc;
}, [imageSrc, matchedRgb, sampleX, sampleY, useVeil]);
return <canvas ref={canvasRef} className="w-full h-full object-cover" />;
};

// Refined ornamental divider
const Ornament = ({ color = c.gold, width = 40 }) => (
    <div className="flex items-center justify-center gap-3 my-3">
        <div style={{ width: `${width}px`, height: '0.5px', background: color, opac
        <div style={{ width: '3px', height: '3px', background: color, transform: 'r
        <div style={{ width: `${width}px`, height: '0.5px', background: color, opac
        </div>
    </div>
);

return (
    <div className="min-h-screen w-full" style={{
        background: c.paper,
        color: c.ink,
        fontFamily: 'Georgia, serif',
    }}>
        <style>{`
            @import url('https://fonts.googleapis.com/css2?family=Cormorant+Garamond:

            @keyframes fadeUp { from { opacity: 0; transform: translateY(8px); } to {
            @keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
            @keyframes scanV { 0%, 100% { transform: translateY(0%); } 50% { transfor
            @keyframes dropIn { 0% { opacity: 0; transform: translateY(-12px); } 100%
            @keyframes pulseRing { 0%, 100% { box-shadow: 0 0 0 0 rgba(255, 61, 127,
            @keyframes shimmer { 0%, 100% { opacity: 0.5; } 50% { opacity: 1; } }
            @keyframes gentleFloat { 0%, 100% { transform: translateY(0); } 50% { tra

            .fade-up { animation: fadeUp 1s cubic-bezier(0.22, 1, 0.36, 1) forwards;
            .fade-in { animation: fadeIn 1.4s ease-out forwards; }
            .scan-line-anim { animation: scanV 2.4s ease-in-out infinite; }
            .pulse-ring-anim { animation: pulseRing 2.6s ease-out infinite; }
            .shimmer-anim { animation: shimmer 2.8s ease-in-out infinite; }
            .float-anim { animation: gentleFloat 3.5s ease-in-out infinite; }

            .display {
                font-family: 'Cormorant Garamond', 'Didot', Georgia, serif;

```

```
    font-weight: 300;
    letter-spacing: 0.005em;
}
.display-italic {
    font-family: 'Cormorant Garamond', Georgia, serif;
    font-style: italic;
    font-weight: 300;
}
.body {
    font-family: 'Inter', 'Helvetica Neue', sans-serif;
    font-weight: 300;
}
.label {
    font-family: 'Inter', sans-serif;
    font-weight: 400;
    letter-spacing: 0.4em;
    text-transform: uppercase;
}
.label-tight {
    font-family: 'Inter', sans-serif;
    font-weight: 400;
    letter-spacing: 0.22em;
    text-transform: uppercase;
}
input { outline: none !important; }
input:focus { border-color: ${c.gold} !important; }

.btn-primary {
    background: ${c.ink};
    color: ${c.paper};
    font-family: 'Inter', sans-serif;
    font-size: 10px;
    letter-spacing: 0.4em;
    text-transform: uppercase;
    font-weight: 400;
    padding: 1.4rem 1.5rem;
    width: 100%;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 0.875rem;
    transition: all 0.5s cubic-bezier(0.22, 1, 0.36, 1);
    border: none;
    cursor: pointer;
}
.btn-primary:hover {
    background: ${c.inkDeep};
```



```

    gap: 1.4rem;
    letter-spacing: 0.42em;
}
.btn-primary:disabled {
    background: ${c.silver};
    cursor: not-allowed;
}

.btn-gold {
    background: ${c.gold};
    color: ${c.paper};
    font-family: 'Inter', sans-serif;
    font-size: 10px;
    letter-spacing: 0.4em;
    text-transform: uppercase;
    font-weight: 500;
    padding: 1.25rem 1.5rem;
    width: 100%;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 0.875rem;
    transition: all 0.5s cubic-bezier(0.22, 1, 0.36, 1);
    border: none;
    cursor: pointer;
}
.btn-gold:hover { background: ${c.goldHover}; gap: 1.4rem; }

.btn-ghost {
    color: ${c.pewter};
    font-family: 'Inter', sans-serif;
    font-size: 9px;
    letter-spacing: 0.34em;
    text-transform: uppercase;
    background: transparent;
    border: none;
    cursor: pointer;
    padding: 0.875rem;
    font-weight: 400;
    transition: color 0.3s;
}
.btn-ghost:hover { color: ${c.ink}; }

.hairline { background: ${c.bone}; height: 0.5px; }

.option-card {
    padding: 1.125rem 1.25rem;

```

```

    border: 0.5px solid ${c.bone};
    background: ${c.paper};
    cursor: pointer;
    transition: all 0.4s cubic-bezier(0.22, 1, 0.36, 1);
    display: flex;
    align-items: center;
    gap: 1rem;
  }
  .option-card:hover {
    border-color: ${c.silver};
    background: ${c.paperShadow};
  }
  .option-card.selected {
    border-color: ${c.gold};
    background: ${c.paperShadow};
    box-shadow: 0 0 0 1px ${c.gold} inset;
  }
`</style>

{/* Persistent file input – always available regardless of screen */}
<input
  ref={fileInputRef}
  type="file"
  accept="image/*"
  capture="environment"
  onChange={handleCapture}
  style={{ display: 'none' }}
/>

{/* ===== LANDING ===== */}
{screen === 'landing' && (
  <div className="min-h-screen flex flex-col px-7 py-10 max-w-md mx-auto fa
    {/* Top bar */}
    <div className="flex justify-between items-center mb-16">
      <div className="label" style={{ color: c.pewter, fontSize: '8px' }}>M
      <div className="label" style={{ color: c.pewter, fontSize: '8px' }}>C
    </div>

    {/* Centred wordmark */}
    <div className="text-center mb-2">
      <div className="flex justify-center mb-8 float-anim">
        <MHMark size={48} color={c.ink} strokeWidth={0.7} />
      </div>

      <h1 className="display" style={{
        fontSize: '46px',
        letterSpacing: '0.24em',

```

```

        color: c.ink,
        lineHeight: 1,
        fontWeight: 300,
    }}>
    MAISON
</h1>
<h1 className="display" style={{
    fontSize: '46px',
    letterSpacing: '0.24em',
    color: c.ink,
    lineHeight: 1,
    fontWeight: 300,
    marginTop: '8px',
}}>
    HUE
</h1>

<Ornament color={c.gold} width={32} />

<div className="label" style={{ color: c.pewter, fontSize: '8.5px', m
    AI · MATCHED · BEAUTY · COLOUR
</div>
</div>

<div className="flex-1 flex flex-col justify-end pb-2">
    {/* Editorial copy – minimal, considered */}
    <div className="mb-14 text-center px-4">
        <p className="display-italic mb-5" style={{
            fontSize: '26px',
            lineHeight: '1.3',
            color: c.ink,
        }}>
            Your colour,<br />intelligently personalised.
        </p>
        <div className="hairline mx-auto mb-5" style={{ width: '32px' }} />
        <p className="body leading-relaxed" style={{
            fontSize: '12.5px',
            color: c.pewter,
            lineHeight: '1.85',
            letterSpacing: '0.01em',
            fontWeight: 300,
        }}>
            The first beauty house where artificial intelligence reads your e
        </p>
    </div>

    {/* Refined spectrum strip */}

```

```

<div className="mb-14">
  <div className="h-1 overflow-hidden" style={{
    background: 'linear-gradient(90deg, #f0ebe0 0%, #d4c4a8 14%, #c4a
    boxShadow: '0 1px 8px rgba(26, 23, 21, 0.06)',
  }} />
  <div className="flex justify-between label mt-4" style={{ color: c.
    <span>PERLE</span>
    <span>MIEL</span>
    <span>SABLE</span>
    <span>ONYX</span>
  </div>
</div>

<button onClick={() => setScreen('intro')} className="btn-primary mb-
  Discover Your Shade <ArrowRight className="w-3.5 h-3.5" strokeWidth
</button>

<button onClick={() => setScreen('waitlist')} className="btn-ghost fl
  <Mail className="w-3 h-3" strokeWidth={1.2} /> Reserve Your Place
</button>

<div className="hairline mt-12 mb-5" />
<div className="text-center label" style={{ color: c.pewter, fontSize
  HAND-BLENDED IN THE COTSWOLDS · LONDON
</div>
</div>
</div>
))

{/* ===== INTRO ===== */}
{screen === 'intro' && (
  <div className="min-h-screen flex flex-col px-7 py-10 max-w-md mx-auto fa
    <div className="flex justify-between items-center mb-14">
      <MhMark size={22} color={c.pewter} strokeWidth={0.7} />
      <button onClick={() => setScreen('landing')} className="p-1">
        <X className="w-3.5 h-3.5" style={{ color: c.pewter }} strokeWidth=
      </button>
    </div>

    <div className="label mb-7" style={{ color: c.gold, fontSize: '8.5px' }
      THE METHOD
    </div>

    <h2 className="display mb-5" style={{
      fontSize: '40px',
      lineHeight: '1.05',
      color: c.ink,

```

```

}}>
  How AI reads<br /><em className="display-italic">your</em> shade.
</h2>

<Ornament color={c.gold} width={32} />

<p className="body mt-6 mb-12" style={{
  fontSize: '12.5px',
  color: c.pewter,
  lineHeight: '1.8',
  letterSpacing: '0.01em',
}}>
  Real computer vision in your browser. Your photograph never leaves yo
</p>

<div className="space-y-9 mb-12">
  {[
    { n: 'I', t: 'Capture', d: 'Upload a clear photograph of the roots
    { n: 'II', t: 'Aim', d: 'Indicate the precise tone. Our matching sy
    { n: 'III', t: 'Match', d: 'K-means clustering and LAB colour-space
  ].map((step, i) => (
    <div key={i} className="flex gap-6 fade-up" style={{ animationDelay
      <div className="display" style={{
        fontSize: '32px',
        color: c.gold,
        fontWeight: 300,
        lineHeight: '1',
        minWidth: '40px',
        letterSpacing: '0.04em',
      }}>
        {step.n}
      </div>
      <div className="flex-1 pt-1">
        <div className="display mb-2.5" style={{ fontSize: '22px', font
          {step.t}
        </div>
        <div className="body" style={{ fontSize: '12.5px', color: c.pew
          {step.d}
        </div>
      </div>
    </div>
  )))
</div>

<div className="hairline mb-6" />

<div className="flex items-start gap-4 mb-12">

```

```

        <Plus className="w-3 h-3 mt-1 flex-shrink-0" style={{ color: c.gold }
        <div className="body" style={{ fontSize: '11px', color: c.pewter, lin
            <span style={{ color: c.ink, fontWeight: 500 }}>Privacy assured.</s
        </div>
    </div>

    <button onClick={() => setScreen('hairInfo')} className="btn-primary">
        Begin <ArrowRight className="w-3.5 h-3.5" strokeWidth={1.2} />
    </button>
</div>
)}

{/* ===== NEW: HAIR INFO ===== */}
{screen === 'hairInfo' && (
    <div className="min-h-screen flex flex-col px-7 py-10 max-w-md mx-auto fa
        <div className="flex justify-between items-center mb-12">
            <MHMark size={22} color={c.pewter} strokeWidth={0.7} />
            <div className="label" style={{ color: c.pewter, fontSize: '8px' }}>I
        </div>

        <h2 className="display mb-4" style={{ fontSize: '36px', lineHeight: '1.
            Tell us your <em className="display-italic">roots</em>.
        </h2>

        <Ornament color={c.gold} width={32} />

        <p className="body mt-5 mb-10" style={{ fontSize: '12.5px', color: c.pe
            So we may match you precisely. One quick question – our AI will read
        </p>

        {/* Roots question – only one we ask */}
        <div className="mb-10">
            <div className="label mb-5" style={{ color: c.ink, fontSize: '9px' }}
                YOUR ROOTS ARE
            </div>
            <div className="space-y-2.5">
                {rootOptions.map((opt) => (
                    <div
                        key={opt.id}
                        onClick={() => setRootShade(opt.id)}
                        className={`option-card ${rootShade === opt.id ? 'selected' : '
                    >
                    {opt.hex ? (
                        <div className="w-7 h-7 rounded-full flex-shrink-0" style={{
                            background: opt.hex,
                            boxShadow: `0 2px 8px ${opt.hex}50, 0 0 0 0.5px ${c.bone}`,
                        }} />

```

```

    ) : (
      <div className="w-7 h-7 rounded-full flex-shrink-0 flex items
        border: `0.5px dashed ${c.silver}` ,
      >>
      <span className="display" style={{ fontSize: '14px', color:
      </div>
    )}
    <div className="flex-1">
      <div className="body" style={{ fontSize: '13px', color: c.ink
        {opt.sub && <div className="body" style={{ fontSize: '11px',
      </div>
      {rootShade === opt.id && <Check className="w-3.5 h-3.5 flex-shr
    </div>
    )}}
  </div>
</div>

<button
  onClick={proceedFromHairInfo}
  disabled={!rootShade}
  className="btn-primary"
>
  Continue <ArrowRight className="w-3.5 h-3.5" strokeWidth={1.2} />
</button>
</div>
)}

{ /* ===== PHOTO GUIDE ===== */ }
{screen === 'photoGuide' && (
  <div className="min-h-screen flex flex-col px-7 py-10 max-w-md mx-auto fa
    <div className="flex justify-between items-center mb-12">
      <MHMark size={22} color={c.pewter} strokeWidth={0.7} />
      <button onClick={() => setScreen('hairInfo')} className="p-1">
        <X className="w-3.5 h-3.5" style={{ color: c.pewter }} strokeWidth=
      </button>
    </div>

    <div className="label mb-4" style={{ color: c.gold, fontSize: '9px' }}>
      II · THE PHOTOGRAPH
    </div>
    <h2 className="display mb-3" style={{ fontSize: '36px', lineHeight: '1.
      How to <em className="display-italic">photograph</em>.
    </h2>
    <Ornament color={c.gold} width={32} />

    <p className="body mt-5 mb-8" style={{ fontSize: '12.5px', color: c.pew
      For the most precise match, please follow these guidelines.

```

</p>

```
{/* Visual diagram of where to photograph */}
<div className="mb-8 p-6 flex flex-col items-center" style={{
  background: c.paperShadow,
  border: `0.5px solid ${c.bone}`,
}}>
  {/* Stylised illustration */}
  <svg width="160" height="180" viewBox="0 0 160 180" fill="none" style
    {/* Head silhouette */}
    <ellipse cx="80" cy="90" rx="55" ry="68" stroke={c.pewter} strokeWi
    {/* Hair top - natural roots */}
    <path d="M 30 60 Q 80 22 130 60 L 130 80 Q 80 60 30 80 Z" fill={c.i
    {/* Hair lengths (lighter) */}
    <path d="M 30 80 Q 80 60 130 80 L 130 155 Q 80 175 30 155 Z" fill={
    {/* Parting line */}
    <line x1="80" y1="22" x2="80" y2="100" stroke={c.paper} strokeWidth
    {/* Highlight circle on root area */}
    <circle cx="80" cy="65" r="22" stroke={c.pop} strokeWidth="2" fill=
    <circle cx="80" cy="65" r="3" fill={c.pop} />
    {/* Arrow pointing to root */}
    <line x1="40" y1="30" x2="65" y2="58" stroke={c.gold} strokeWidth="
    <polygon points="65,58 60,52 56,58" fill={c.gold} />
    {/* "X" mark on the lengths */}
    <g transform="translate(120, 130)">
      <circle cx="0" cy="0" r="14" fill={c.paper} stroke={c.pewter} str
      <line x1="-5" y1="-5" x2="5" y2="5" stroke={c.pewter} strokeWidth
      <line x1="5" y1="-5" x2="-5" y2="5" stroke={c.pewter} strokeWidth
    </g>
  </svg>

  <div className="display-italic text-center" style={{ fontSize: '15px'
    Photograph along your parting,<br />close to the scalp.
  </div>
</div>

{/* Three concise guidelines */}
<div className="space-y-5 mb-10">
  <div className="flex items-start gap-4">
    <div className="display" style={{ fontSize: '14px', color: c.gold,
    <div>
      <div className="body" style={{ fontSize: '12.5px', color: c.ink,
      <div className="body" style={{ fontSize: '11.5px', color: c.pewte
        Stand near a window. Avoid yellow indoor lighting.
      </div>
    </div>
  </div>
</div>
```



```

    <div className="flex items-start gap-4">
      <div className="display" style={{ fontSize: '14px', color: c.gold,
      <div>
        <div className="body" style={{ fontSize: '12.5px', color: c.ink,
        <div className="body" style={{ fontSize: '11.5px', color: c.pewte
          Capture the line where natural roots meet dyed length – not the
        </div>
      </div>
    </div>
    <div className="flex items-start gap-4">
      <div className="display" style={{ fontSize: '14px', color: c.gold,
      <div>
        <div className="body" style={{ fontSize: '12.5px', color: c.ink,
        <div className="body" style={{ fontSize: '11.5px', color: c.pewte
          Hold the camera 20–30 cm away. Tap to focus before capturing.
        </div>
      </div>
    </div>
  </div>

  <button onClick={startCapture} className="btn-primary">
    <Camera className="w-3.5 h-3.5" strokeWidth={1.2} /> Take or Upload P
  </button>
</div>
)}

{/* ===== NEW: ATELIER SYSTEM INTRO ===== */}
{screen === 'atelierIntro' && (
  <div className="min-h-screen flex flex-col px-7 py-10 max-w-md mx-auto fa
    <div className="flex justify-between items-center mb-12">
      <MHMark size={22} color={c.pewter} strokeWidth={0.7} />
      <button onClick={() => setScreen('hairInfo')} className="p-1">
        <X className="w-3.5 h-3.5" style={{ color: c.pewter }} strokeWidth=
      </button>
    </div>

    <div className="label mb-4" style={{ color: c.gold, fontSize: '9px' }}>
      BY DESIGN · FOR YOU
    </div>

    <h2 className="display mb-5" style={{ fontSize: '40px', lineHeight: '1.
      The <em className="display-italic">Atelier</em><br />System.
    </h2>

    <Ornament color={c.gold} width={32} />

    <p className="body mt-6 mb-10" style={{ fontSize: '12.5px', color: c.pe
      We've designed a three-step bespoke ritual – engineered for women who

```

</p>

```
{/* Three-step illustration */}
<div className="space-y-5 mb-10">
  <div className="p-6" style={{ background: c.paperShadow, border: `0.5
    <div className="flex items-start gap-5">
      <div className="display" style={{ fontSize: '28px', color: c.gold
        <div className="flex-1">
          <div className="display mb-2" style={{ fontSize: '20px', color:
            <div className="body" style={{ fontSize: '12px', color: c.pewte
              A featherlight whitening base. Neutralises depth and prepares
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

  <div className="p-6" style={{ background: c.paperShadow, border: `0.5
    <div className="flex items-start gap-5">
      <div className="display" style={{ fontSize: '28px', color: c.gold
        <div className="flex-1">
          <div className="display mb-2" style={{ fontSize: '20px', color:
            <div className="body" style={{ fontSize: '12px', color: c.pewte
              Your bespoke colour, formulated from your photograph. Seamless
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

  <div className="p-6" style={{ background: c.paperShadow, border: `0.5
    <div className="flex items-start gap-5">
      <div className="display" style={{ fontSize: '28px', color: c.gold
        <div className="flex-1">
          <div className="display mb-2" style={{ fontSize: '20px', color:
            <div className="body" style={{ fontSize: '12px', color: c.pewte
              A weightless setting mist. Locks colour against transfer, res
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

<div className="hairline mb-6" />

<div className="flex items-start gap-4 mb-10">
  <Plus className="w-3 h-3 mt-1 flex-shrink-0" style={{ color: c.gold }
  <div className="body" style={{ fontSize: '11px', color: c.pewter, lin
    The Atelier System is the world's first complete bespoke root conce
```

```

    </div>
  </div>

  <button onClick={startCapture} className="btn-primary">
    <Camera className="w-3.5 h-3.5" strokeWidth={1.2} /> Continue to Phot
  </button>
</div>
))

{/* ===== AIM ===== */}
{screen === 'aim' && (
  <div className="min-h-screen flex flex-col px-7 py-10 max-w-md mx-auto fa
    <div className="flex justify-between items-center mb-10">
      <MHMark size={22} color={c.pewter} strokeWidth={0.7} />
      <div className="label" style={{ color: c.pewter, fontSize: '8px' }}>
        III · AIM
      </div>
    </div>

    <h2 className="display mb-3" style={{ fontSize: '36px', lineHeight: '1.
      Tap your <em className="display-italic">roots</em>.
    </h2>

    <Ornament color={c.gold} width={32} />

    <p className="body mt-4 mb-5" style={{ fontSize: '12.5px', color: c.pew
      Hair has dimension — light strands, deeper shadows, midtones. Tap up
    </p>

    {/* Visual instruction guide */}
    <div className="mb-6 p-4" style={{ background: c.paperShadow, border: `
      <div className="flex items-start gap-3">
        <div className="flex-shrink-0 mt-0.5">
          <div className="w-5 h-5 rounded-full flex items-center justify-ce
            <span className="display" style={{ color: c.paper, fontSize: '1
          </div>
        </div>
        <div className="body" style={{ fontSize: '11px', color: c.pewter, l
          <strong style={{ color: c.ink, fontWeight: 500 }}>For dimensional
        </div>
      </div>
    </div>

    {capturedImage && (
      <div
        className="relative overflow-hidden mb-5 cursor-crosshair"
        style={{
          aspectRatio: '3/4',
          boxShadow: '0 16px 48px rgba(26, 23, 21, 0.12)',

```

```

    }}
    onClick={ (e) => {
      if (samplePoints.length >= 3) return; // max 3 points
      const rect = e.currentTarget.getBoundingClientRect();
      const x = ((e.clientX - rect.left) / rect.width) * 100;
      const y = ((e.clientY - rect.top) / rect.height) * 100;
      setSamplePoints([...samplePoints, { x, y }]);
    }}
  >
  <img ref={imgRef} src={capturedImage} alt="captured" crossOrigin="a

  {/* Hint label when no points tapped yet */}
  {samplePoints.length === 0 && (
    <div className="absolute top-3 left-3 right-3 px-3 py-2 pointer-e
      background: 'rgba(26, 23, 21, 0.78)',
      backdropFilter: 'blur(6px)',
    >
    <div className="label text-center" style={{ color: c.paper, fon
      TAP THE ROOT AREA
    </div>
  </div>
  )}

  {/* Render all crosshairs with numbers */}
  {samplePoints.map((pt, idx) => (
    <div
      key={idx}
      className="absolute pointer-events-none pulse-ring-anim"
      style={{
        left: `${pt.x}%`,
        top: `${pt.y}%`,
        transform: 'translate(-50%, -50%)',
        width: 64,
        height: 64,
        border: `2px solid ${c.pop}`,
        borderRadius: '50%',
        boxShadow: `0 0 0 1px rgba(255,255,255,0.9), 0 0 16px ${c.pop}
        animationDelay: `${idx * 0.4}s`,
      }}
    >
    <div className="absolute top-1/2 left-0 right-0" style={{ heigh
    <div className="absolute left-1/2 top-0 bottom-0" style={{ widt
    {/* Number badge in center */}
    <div
      className="absolute top-1/2 left-1/2 -translate-x-1/2 -transl
      style={{
        width: 18,

```

```

        height: 18,
        borderRadius: '50%',
        background: c.pop,
        boxShadow: `0 0 8px ${c.pop}, 0 0 0 2px white`,
      }}
    >
      <span className="display" style={{ color: 'white', fontSize:
        {idx + 1}
      </span>
    </div>
  </div>
  )}}
</div>
)}

{/* Status row - points count and clear */}
{capturedImage && (
  <div className="flex items-center justify-between mb-6 px-1">
    <div className="label" style={{ color: c.pewter, fontSize: '9px' }}
      {samplePoints.length === 0
        ? 'TAP TO BEGIN'
        : samplePoints.length === 3
          ? '3 OF 3 POINTS · MAXIMUM REACHED'
          : `${samplePoints.length} OF 3 POINTS · TAP FOR MORE PRECISIO
    </div>
    {samplePoints.length > 0 && (
      <button
        onClick={() => setSamplePoints([])}
        className="label"
        style={{
          color: c.gold,
          fontSize: '9px',
          background: 'transparent',
          border: 'none',
          cursor: 'pointer',
          padding: 0,
        }}
      >
        CLEAR
      </button>
    )}
  </div>
)}

<button
  onClick={runRealAnalysis}
  disabled={samplePoints.length === 0}

```

```

        className="btn-primary mb-3"
      >
        <Crosshair className="w-3.5 h-3.5" strokeWidth={1.2} /> Analyse My Sh
      </button>
      <button onClick={() => { setCapturedImage(null); setSamplePoints([]); s
        Choose a different photograph
      </button>
    </div>
  )}

  {/* ===== ANALYZE ===== */}
  {screen === 'analyze' && (
    <div className="min-h-screen flex flex-col px-7 py-10 max-w-md mx-auto">
      <div className="flex justify-between items-center mb-10">
        <MHMark size={22} color={c.pewter} strokeWidth={0.7} />
        <div className="label shimmer-anim" style={{ color: c.gold, fontSize:
          ANALYSING
        </div>
      </div>
    </div>

    <div className="relative overflow-hidden mb-10" style={{
      aspectRatio: '3/4',
      boxShadow: '0 16px 48px rgba(26, 23, 21, 0.12)',
    }}>
      {capturedImage && <img ref={imgRef} src={capturedImage} alt="captured
    <div className="absolute inset-0" style={{ background: 'linear-gradient
    {analyzing && (
      <>
        <div className="absolute left-0 right-0 scan-line-anim" style={{
          height: '1px',
          background: `linear-gradient(90deg, transparent 0%, ${c.gold} 5
          boxShadow: `0 0 16px ${c.gold}``,
          top: 0,
        > />
        {/* Render all sample points */}
        {samplePoints.map((pt, idx) => (
          <div
            key={idx}
            className="absolute pointer-events-none"
            style={{
              left: `${pt.x}%`,
              top: `${pt.y}%`,
              transform: 'translate(-50%, -50%)',
              width: 64,
              height: 64,
              border: `2px solid ${c.pop}`,
              borderRadius: '50%',

```

```

        boxShadow: `0 0 0 1px rgba(255,255,255,0.9), 0 0 24px ${c.p
    }}
  />
  )))
  <div className="absolute inset-0">
    <div className="absolute top-3 left-3" style={{ width: '14px',
    <div className="absolute top-3 right-3" style={{ width: '14px',
    <div className="absolute bottom-3 left-3" style={{ width: '14px
    <div className="absolute bottom-3 right-3" style={{ width: '14p
  </div>
  </>
  )}
</div>

<div className="space-y-4 mb-10">
  {analysisStages.map((stage, i) => (
    <div key={i} className="flex items-center gap-4" style={{ opacity:
      {i < analysisStep ? <Check className="w-3 h-3 flex-shrink-0" styl
        : i === analysisStep ? <Loader2 className="w-3 h-3 flex-shrink-
        : <div className="w-3 h-3 rounded-full border flex-shrink-0" st
    <div className="label-tight" style={{
      color: i <= analysisStep ? c.ink : c.pewter,
      fontSize: '10px',
    }}>
      {stage}
    </div>
  </div>
  )})
</div>

<Ornament color={c.gold} width={32} />

<div className="text-center display-italic mt-4" style={{ fontSize: '17
  Reading your pigment...
</div>
</div>
)}

{/* ===== PREVIEW ===== */}
{screen === 'preview' && matchedShade && capturedImage && (
  <div className="min-h-screen flex flex-col px-7 py-10 max-w-md mx-auto fa
  <div className="flex justify-between items-center mb-8">
    <MHMark size={22} color={c.pewter} strokeWidth={0.7} />
    <div className="label" style={{ color: c.pewter, fontSize: '8px' }}>
      IV · PREVIEW
    </div>
  </div>
</div>

```

```

<div className="label mb-3" style={{ color: c.gold, fontSize: '9px' }}>
  N° {matchedShade.formulaNum} · {matchedShade.confidence}% CONFIDENCE
</div>
<h2 className="display mb-2" style={{ fontSize: '40px', lineHeight: '1.
  How <em className="display-italic">{matchedShade.name}</em><br />look
</h2>
<Ornament color={c.gold} width={32} />

<p className="body mt-5 mb-8" style={{ fontSize: '12.5px', color: c.pew
  Your matched colour applied to the roots in your photograph. Does thi
</p>

{/* The painted preview */}
<div className="relative overflow-hidden mb-6" style={{
  aspectRatio: '3/4',
  boxShadow: '0 16px 48px rgba(26, 23, 21, 0.15)',
  border: `0.5px solid ${c.bone}`,
  background: c.paperShadow,
}}>
  <PreviewCanvas
    imageSrc={capturedImage}
    matchedRgb={matchedShade.rgb}
    sampleX={samplePoints.length > 0 ? samplePoints.reduce((a, p) => a
    sampleY={samplePoints.length > 0 ? samplePoints.reduce((a, p) => a
  />
  {/* Watermark badge */}
  <div className="absolute top-3 right-3 px-3 py-1.5" style={{
    background: 'rgba(26, 23, 21, 0.75)',
    backdropFilter: 'blur(8px)',
  }}>
    <div className="label" style={{ color: c.paper, fontSize: '7px' }}>
      MAISON HUE · PREVIEW
    </div>
  </div>
</div>

{/* Shade summary card */}
<div className="flex items-center gap-4 p-4 mb-8" style={{
  background: c.paperShadow,
  border: `0.5px solid ${c.bone}`,
}}>
  <div className="w-12 h-12 rounded-full flex-shrink-0" style={{
    background: matchedShade.hex,
    boxShadow: `0 4px 16px ${matchedShade.hex}80, 0 0 0 0.5px ${c.gold}
  }} />
  <div className="flex-1">

```



```

    <div className="display-italic" style={{ fontSize: '20px', color: c
      {matchedShade.name}
    </div>
    <div className="body mt-1" style={{ fontSize: '11px', color: c.pewt
      {matchedShade.hex.toUpperCase()} · ΔE {matchedShade.deltaE}
    </div>
  </div>
</div>

{/* Action buttons — adapted to match confidence */}
{parseFloat(matchedShade.deltaE) < 10 ? (
  // High confidence — clean approve / retake
  <>
    <button onClick={() => setScreen('result')} className="btn-primary
      Approve Match <ArrowRight className="w-3.5 h-3.5" strokeWidth={1.
    </button>
    <button onClick={() => { setCapturedImage(null); setMatchedShade(nu
      Try Another Photograph
    </button>
  </>
) : parseFloat(matchedShade.deltaE) < 20 ? (
  // Medium confidence — offer adjust
  <>
    <p className="body text-center mb-5" style={{ fontSize: '11.5px', c
      Our match is close. Would you like to refine, or does this look r
    </p>
    <button onClick={() => setScreen('result')} className="btn-primary
      Approve Match <ArrowRight className="w-3.5 h-3.5" strokeWidth={1.
    </button>
    <div className="flex gap-2 mb-3">
      <button
        onClick={() => {
          // Slightly cooler — shift LAB b- and rerun shade name
          const newRgb = [
            Math.max(0, matchedShade.rgb[0] - 12),
            Math.max(0, matchedShade.rgb[1] - 4),
            Math.min(255, matchedShade.rgb[2] + 8),
          ];
          const newLab = rgbToLab(newRgb[0], newRgb[1], newRgb[2]);
          const newUndertone = getUndertone(newLab);
          const { recipe: newRecipe, deltaE: newDE } = generateRecipe(n
          const newName = getShadeName(newLab, newUndertone, newRgb);
          const newHex = '#' + newRgb.map(v => v.toString(16).padStart(
          setMatchedShade({
            ...matchedShade,
            name: newName, hex: newHex, rgb: newRgb, lab: newLab,
            undertone: newUndertone, fullRecipe: newRecipe,

```

```

        confidence: getConfidence(newDE), deltaE: newDE.toFixed(2),
        desc: getDescription(newName),
    });
  }}
  className="flex-1 py-3"
  style={{
    background: 'transparent',
    border: `0.5px solid ${c.silver}`,
    color: c.ink,
    fontSize: '9px',
    letterSpacing: '0.3em',
    textTransform: 'uppercase',
    fontFamily: 'Inter, sans-serif',
    cursor: 'pointer',
  }}
>
  Slightly Cooler
</button>
<button
  onClick={() => {
    // Slightly warmer - shift LAB b+ and rerun
    const newRgb = [
      Math.min(255, matchedShade.rgb[0] + 12),
      Math.min(255, matchedShade.rgb[1] + 4),
      Math.max(0, matchedShade.rgb[2] - 8),
    ];
    const newLab = rgbToLab(newRgb[0], newRgb[1], newRgb[2]);
    const newUndertone = getUndertone(newLab);
    const { recipe: newRecipe, deltaE: newDE } = generateRecipe(n
    const newName = getShadeName(newLab, newUndertone, newRgb);
    const newHex = '#' + newRgb.map(v => v.toString(16).padStart(
    setMatchedShade({
      ...matchedShade,
      name: newName, hex: newHex, rgb: newRgb, lab: newLab,
      undertone: newUndertone, fullRecipe: newRecipe,
      confidence: getConfidence(newDE), deltaE: newDE.toFixed(2),
      desc: getDescription(newName),
    });
  }}
  className="flex-1 py-3"
  style={{
    background: 'transparent',
    border: `0.5px solid ${c.silver}`,
    color: c.ink,
    fontSize: '9px',
    letterSpacing: '0.3em',
    textTransform: 'uppercase',

```

```

        fontFamily: 'Inter, sans-serif',
        cursor: 'pointer',
      }}
    >
      Slightly Warmer
    </button>
  </div>
  <button onClick={() => { setCapturedImage(null); setMatchedShade(nu
    Try Another Photograph
  </button>
</>
) : (
  // Low confidence - gracefully ask for better photo
  <>
    <div className="p-5 mb-5" style={{ background: c.paperShadow, borde
      <div className="flex items-center gap-3 mb-2">
        <div style={{ width: '4px', height: '4px', background: c.gold,
          <div className="label" style={{ color: c.gold, fontSize: '8.5px
        </div>
        <div className="body" style={{ fontSize: '11.5px', color: c.pewte
          The lighting in your photograph is making it harder for us to m
        </div>
      </div>
      <button onClick={() => { setCapturedImage(null); setMatchedShade(nu
        <Camera className="w-3.5 h-3.5" strokeWidth={1.2} /> Take Another
      </button>
      <button onClick={() => setScreen('result')} className="btn-ghost">
        Continue Anyway
      </button>
    </>
  )}
</div>
)}

{/* ===== RESULT ===== */}
{screen === 'result' && matchedShade && (
  <div className="min-h-screen flex flex-col px-7 py-10 max-w-md mx-auto fa
    <div className="flex justify-between items-center mb-8">
      <MHMark size={22} color={c.pewter} strokeWidth={0.7} />
      <div className="label" style={{ color: c.pewter, fontSize: '8px' }}>
        YOUR FORMULA
      </div>
    </div>
  </div>

  <div className="label mb-3" style={{ color: c.gold, fontSize: '9px' }}>
    N° {matchedShade.formulaNum} · {matchedShade.confidence}% CONFIDENCE
  </div>

```

```

<h2 className="display mb-2" style={{
  fontSize: '64px',
  lineHeight: '0.95',
  fontWeight: 300,
  color: c.ink,
}}>
  <em className="display-italic">{matchedShade.name}</em>
</h2>
<Ornament color={c.gold} width={32} />

{/* TRY IT ON – press and hold to reveal painted preview */}
{capturedImage && (
  <div className="mb-10 mt-2">
    <div className="label mb-3 text-center" style={{ color: c.gold, fon
      TRY IT ON
    </div>
    <div
      className="relative overflow-hidden select-none"
      style={{
        aspectRatio: '3/4',
        boxShadow: '0 16px 48px rgba(26, 23, 21, 0.15)',
        border: `0.5px solid ${c.bone}`,
        background: c.paperShadow,
        cursor: 'pointer',
        userSelect: 'none',
        WebkitUserSelect: 'none',
        WebkitTouchCallout: 'none',
      }}
      onMouseDown={() => setTryItOnActive(true)}
      onMouseUp={() => setTryItOnActive(false)}
      onMouseLeave={() => setTryItOnActive(false)}
      onTouchStart={(e) => { e.preventDefault(); setTryItOnActive(true)
      onTouchEnd={(e) => { e.preventDefault(); setTryItOnActive(false);
      onTouchCancel={() => setTryItOnActive(false)}
      onContextMenu={(e) => e.preventDefault()}
    >
      {/* Original photo always shown */}
      <img
        src={capturedImage}
        alt="your photograph"
        className="w-full h-full object-cover absolute inset-0 pointer-
        style={{ opacity: tryItOnActive ? 0 : 1, transition: 'opacity 0
        draggable={false}
      />
      {/* Preview canvas – shown on press */}
      <div
        className="absolute inset-0 pointer-events-none"

```

```

        style={{ opacity: tryItOnActive ? 1 : 0, transition: 'opacity 0
>
<PreviewCanvas
  imageSrc={capturedImage}
  matchedRgb={matchedShade.rgb}
  sampleX={samplePoints.length > 0 ? samplePoints.reduce((a, p)
  sampleY={samplePoints.length > 0 ? samplePoints.reduce((a, p)
  useVeil={needsAtelierSystem}
  />
</div>

{/* Hint label – appears when not active */}
{!tryItOnActive && (
  <div className="absolute bottom-4 left-4 right-4 flex justify-c
    <div className="px-4 py-2.5" style={{
      background: 'rgba(26, 23, 21, 0.78)',
      backdropFilter: 'blur(8px)',
    }}>
      <div className="label" style={{ color: c.paper, fontSize: '
        PRESS & HOLD TO PREVIEW
      </div>
    </div>
  </div>
)}

{/* Active label – appears when pressed */}
{tryItOnActive && (
  <div className="absolute top-4 left-4 right-4 flex justify-betw
    <div className="px-3 py-1.5" style={{
      background: 'rgba(26, 23, 21, 0.78)',
      backdropFilter: 'blur(8px)',
    }}>
      <div className="label" style={{ color: c.paper, fontSize: '
        {needsAtelierSystem ? 'VEIL + HUE APPLIED' : 'WITH MAISON
      </div>
    </div>
    <div className="px-3 py-1.5" style={{
      background: matchedShade.hex,
    }}>
      <div className="label" style={{ color: c.paper, fontSize: '
        N° {matchedShade.formulaNum}
      </div>
    </div>
  </div>
)}
</div>
<p className="body text-center mt-3" style={{ fontSize: '11px', col

```

```

        {needsAtelierSystem
          ? `Hold the photograph to see The Veil and ${matchedShade.name}
          : `Hold the photograph to see ${matchedShade.name} applied to y
        </p>
      </div>
    )}

```

```

    {/* Premium shade orb */}
    <div className="relative mb-12 mt-2" style={{ animation: 'dropIn 1s cub
      <div className="aspect-square relative overflow-hidden" style={{
        background: `radial-gradient(circle at 32% 30%, ${matchedShade.hex}
        boxShadow: `0 32px 64px -20px ${matchedShade.hex}90, 0 0 0 0.5px ${
      }}>

```

```

      <div className="absolute inset-0" style={{ background: 'radial-grad

```

```

    {/* Top branding on swatch */}

```

```

    <div className="absolute top-5 left-5 right-5 flex justify-between

```

```

      <MHMark size={18} color={'#fbfaf6aa'} strokeWidth={0.5} />

```

```

      <div className="text-right">

```

```

        <div className="label" style={{ color: '#fbfaf6aa', fontSize: '

```

```

        <div className="display-italic mt-1" style={{ color: '#fbfaf6',
          N° {matchedShade.formulaNum}

```

```

        </div>

```

```

      </div>

```

```

    </div>

```

```

    {/* Bottom data */}

```

```

    <div className="absolute bottom-5 left-5 right-5 flex justify-betwe

```

```

      <div>

```

```

        <div className="label mb-1" style={{ color: '#fbfaf6aa', fontSi

```

```

        <div className="body" style={{ color: '#fbfaf6', fontFamily: 'C
          {matchedShade.lab.L} · {matchedShade.lab.a} · {matchedShade.l

```

```

        </div>

```

```

      </div>

```

```

      <div className="text-right">

```

```

        <div className="label mb-1" style={{ color: '#fbfaf6aa', fontSi

```

```

        <div className="body" style={{ color: '#fbfaf6', fontFamily: 'C
          {matchedShade.hex.toUpperCase()}

```

```

        </div>

```

```

      </div>

```

```

    </div>

```

```

  </div>

```

```

</div>

```

```

    {/* Atelier System badge if applicable */}

```

```

    {needsAtelierSystem && (

```

```

      <div className="mb-10 p-5" style={{ background: c.paperShadow, border

```

```

    <div className="flex items-center gap-3 mb-2">
      <div style={{ width: '4px', height: '4px', background: c.gold, tr
        <div className="label" style={{ color: c.gold, fontSize: '8.5px'
      </div>
    <div className="body" style={{ fontSize: '11.5px', color: c.pewter,
      Your formula ships as a complete ritual – <em style={{ color: c.i
    </div>
  </div>
)}

{/* Recipe – refined editorial */}
<div className="mb-12">
  <div className="hairline mb-5" />
  <div className="flex items-center justify-between mb-5">
    <div className="label" style={{ color: c.pewter, fontSize: '8.5px'
    <div className="label" style={{ color: c.gold, fontSize: '8.5px' }}
  </div>
  <div className="flex gap-px mb-6 h-2 overflow-hidden">
    {matchedShade.fullRecipe.map((p, i) => (
      <div key={i} style={{ background: p.hex, width: `${p.pct}%` }} />
    ))}
  </div>
  <div className="space-y-3.5">
    {matchedShade.fullRecipe.map((p, i) => (
      <div key={i} className="flex justify-between items-center">
        <div className="flex items-center gap-3">
          <div className="w-2.5 h-2.5 rounded-full" style={{ background
            <span className="body" style={{ color: c.ink, fontSize: '12px
          </div>
          <div className="body" style={{ fontFamily: 'Courier, monospace'
            {p.pct}%
          </div>
        </div>
      </div>
    ))}
  </div>
  <div className="hairline mt-6" />
</div>

<p className="display-italic mb-4" style={{
  fontSize: '17px',
  color: c.pewter,
  lineHeight: '1.5',
  fontWeight: 300,
}}>
  Hand-blended in our atelier.
</p>

```

```

{ /* Three-step ritual list - discreet but present */}
<div className="mb-12">
  <div className="hairline mb-4" />
  <div className="label mb-4" style={{ color: c.pewter, fontSize: '8.5p
  <div className="space-y-3">
    <div className="flex items-baseline gap-3">
      <div className="display" style={{ fontSize: '11px', color: c.gold
      <div className="body flex-1" style={{ fontSize: '12px', color: c.
        <em className="display-italic" style={{ color: c.ink }}>The Vei
        <span style={{ color: c.pewter }}> · whitening base{needsAtelie
      </div>
    </div>
  <div className="flex items-baseline gap-3">
    <div className="display" style={{ fontSize: '11px', color: c.gold
    <div className="body flex-1" style={{ fontSize: '12px', color: c.
      <em className="display-italic" style={{ color: c.ink }}>The Hue
      <span style={{ color: c.pewter }}> · your bespoke colour</span>
    </div>
  </div>
  <div className="flex items-baseline gap-3">
    <div className="display" style={{ fontSize: '11px', color: c.gold
    <div className="body flex-1" style={{ fontSize: '12px', color: c.
      <em className="display-italic" style={{ color: c.ink }}>The Fin
      <span style={{ color: c.pewter }}> · setting mist with shine</s
    </div>
  </div>
  </div>
  <div className="hairline mt-4" />
</div>

{ /* Reserve panel */}
<div className="p-7 mb-5" style={{
  background: c.ink,
  color: c.paper,
  boxShadow: '0 24px 56px rgba(26, 23, 21, 0.25)',
}}>
  <div className="flex items-center justify-between mb-6">
    <MhMark size={20} color={c.gold} strokeWidth={0.6} />
    <div className="label" style={{ color: c.gold, fontSize: '8px' }}>C
  </div>
  <div className="display mb-4" style={{ fontSize: '28px', lineHeight:
    Reserve <em className="display-italic" style={{ color: c.goldHover
  </div>
  <div className="hairline my-5" style={{ background: c.pewter, opacity
  <div className="body mb-6" style={{ fontSize: '12px', lineHeight: '1.
    Founding members receive early-access pricing, priority shipping, t
  </div>

```



```

        <button onClick={() => setScreen('waitlist')} className="btn-gold">
            Reserve My Formula <ArrowRight className="w-3.5 h-3.5" strokeWidth=
        </button>
    </div>

    <button onClick={() => { setScreen('aim'); setMatchedShade(null); setTr
        Try a Different Photograph
    </button>
</div>
)}

{ /* ===== WAITLIST ===== */}
{screen === 'waitlist' && (
    <div className="min-h-screen flex flex-col px-7 py-10 max-w-md mx-auto fa
        {!waitlistJoined ? (
            <>
                <div className="flex justify-between items-center mb-12">
                    <MHMark size={22} color={c.pewter} strokeWidth={0.7} />
                    <button onClick={() => setScreen(matchedShade ? 'result' : 'landi
                        <X className="w-3.5 h-3.5" style={{ color: c.pewter }} strokeWi
                    </button>
                </div>

                <div className="label mb-4" style={{ color: c.gold, fontSize: '9px'
                    FOUNDING MEMBERS · BY INVITATION
                </div>
                <h2 className="display mb-3" style={{ fontSize: '48px', lineHeight:
                    Join the<br /><em className="display-italic">founders</em>.
                </h2>
                <Ornament color={c.gold} width={32} />
                <p className="body mt-5 mb-10" style={{ fontSize: '12.5px', color:
                    Maison Hue is coming soon. Founding members are the first to know
                </p>

                {matchedShade && (
                    <div className="p-6 mb-8" style={{ background: c.paperShadow, bor
                        <div className="flex items-center justify-between mb-4">
                            <div className="label" style={{ color: c.pewter, fontSize: '8
                                <div className="label" style={{ color: c.gold, fontSize: '8.5
                            </div>
                        </div>
                        <div className="flex items-center gap-5">
                            <div className="w-16 h-16 rounded-full flex-shrink-0" style={
                                background: matchedShade.hex,
                                boxShadow: `0 8px 24px ${matchedShade.hex}80, 0 0 0 0.5px $
                            </div>
                        </div>
                            <div className="display-italic" style={{ fontSize: '24px',

```

```

        <div className="body mt-1" style={{ fontSize: '11.5px', col
      </div>
    </div>
  </div>
)}

```

```

<div className="space-y-5 mb-10">
  {[
    { t: 'Early-access pricing', d: 'A founder rate on your first r
    { t: 'Priority shipping', d: 'First to receive when Maison Hue
    { t: 'The complete three-step ritual', d: 'The Veil · The Hue ·
    { t: 'Complimentary calibration card', d: 'Pantone-corrected ·
    { t: 'Founders updates', d: 'Behind-the-scenes previews as we b
  ].map((b, i) => (
    <div key={i} className="flex items-start gap-4">
      <div className="display" style={{ fontSize: '13px', color: c.
        +
      </div>
      <div>
        <div className="body" style={{ fontSize: '13px', color: c.i
          <div className="body" style={{ fontSize: '11.5px', color: c
        </div>
      </div>
    </div>
  )))
</div>

```

```

<div className="hairline mb-6" />

```

```

<input
  type="email"
  placeholder="your@email.com"
  value={email}
  onChange={(e) => setEmail(e.target.value)}
  className="w-full px-5 py-4 mb-4 body"
  style={{
    background: 'transparent',
    border: `0.5px solid ${c.silver}`,
    color: c.ink,
    fontSize: '13px',
    letterSpacing: '0.02em',
  }}
/>
<button onClick={joinWaitlist} className="btn-primary">
  Reserve My Place <ArrowRight className="w-3.5 h-3.5" strokeWidth=
</button>

```

```

<div className="text-center mt-8">

```

```

        <div className="label" style={{ color: c.pewter, fontSize: '8px'
          NO COMMITMENT · UNSUBSCRIBE ANYTIME
        </div>
      </div>
    </>
  ) : (
    <div className="flex-1 flex flex-col justify-center text-center fade-
      <div className="flex justify-center mb-10 float-anim">
        <MHMark size={64} color={c.gold} strokeWidth={0.5} />
      </div>
      <div className="label mb-4" style={{ color: c.gold, fontSize: '9px'
        WELCOME · FOUNDER N° {waitlistCount}
      </div>
      <h2 className="display mb-2" style={{ fontSize: '56px', lineHeight:
        You're <em className="display-italic">in</em>.
      </h2>
      <Ornament color={c.gold} width={32} />
      <p className="body mt-6 mb-12 mx-auto" style={{ fontSize: '12.5px',
        {matchedShade ? <>We've reserved <em style={{ color: c.ink, fontS
        We will be in touch as we approach launch with founder-only previ
      </p>

      {matchedShade && (
        <div className="p-6 mb-12 mx-auto" style={{
          background: c.ink,
          color: c.paper,
          maxWidth: 320,
          boxShadow: '0 24px 48px rgba(26, 23, 21, 0.25)',
        }}>
          <div className="flex items-center gap-4">
            <div className="w-14 h-14 rounded-full flex-shrink-0" style={
              background: matchedShade.hex,
              boxShadow: `0 0 0 0.5px ${c.gold}`,
            }} />
            <div className="text-left">
              <div className="label" style={{ opacity: 0.55, fontSize: '7
                <div className="display-italic" style={{ fontSize: '20px',
              </div>
            </div>
          </div>
        </div>
      )}

    <div className="hairline mb-5 mx-auto" style={{ width: '60px' }} />
    <div className="label mb-5" style={{ color: c.pewter, fontSize: '8p
    <div className="flex justify-center gap-3 mb-12">
      <button className="px-5 py-3" style={{
        border: `0.5px solid ${c.ink}`,

```

```

        color: c.ink,
        fontSize: '8.5px',
        letterSpacing: '0.36em',
        textTransform: 'uppercase',
        fontFamily: 'Inter, sans-serif',
        fontWeight: 400,
        background: 'transparent',
        cursor: 'pointer',
    }}>
        Instagram
    </button>
    <button className="px-5 py-3" style={{
        border: `0.5px solid ${c.ink}`,
        color: c.ink,
        fontSize: '8.5px',
        letterSpacing: '0.36em',
        textTransform: 'uppercase',
        fontFamily: 'Inter, sans-serif',
        fontWeight: 400,
        background: 'transparent',
        cursor: 'pointer',
    }}>
        Copy Link
    </button>
</div>

<button
    onClick={() => {
        setScreen('landing'); setCapturedImage(null); setMatchedShade(n
        setWaitlistJoined(false); setEmail('');
        setRootShade(null); setNeedsAtelierSystem(false); setTryItOnAct
        setSamplePoints([]);
    }}
    className="btn-ghost"
>
    Return to Start
</button>
</div>

    })
</div>

    })
</div>

);
}

```